

Series: Hardening Apache HTTPD in Docker (Part 4 of 4)

Author: H. Brister Wooley

Published: 2026-02-25

Series: Hardening Apache HTTPD in Docker (4/4)

Published Link: <https://github.com/hbwooley/deep-thoughts-and-ramblings/blob/main/2026-02-25-hardening-apache-docker-part4.md>

Here we are – the final piece. We’ve built a non-root container, locked down `httpd.conf` with default-deny rules, and configured headers, TLS, and ModSecurity with surgical exclusions.

But config files are just text until the kernel enforces them. `docker-compose.yml` is where rubber meets runtime – capabilities dropped, networks segmented, volumes mounted read-only. And the rewrite rules? That’s where we wire everything together: host validation, bot blocking, and fail2ban integration that actually works in production.

This is the part most teams skip. They harden the image, deploy with defaults, and wonder why the attacker reached the database. Let’s not be that team.

`docker-compose.yml`: Runtime Hardening

```
services:
  httpd:
    build:
      context: .
      dockerfile: Dockerfile
    image: httpd-hardened:latest
    container_name: httpd-hardened
    user: "48:48"
    read_only: true
    tmpfs:
      - /run/httpd
      - /tmp
      - /var/cache/httpd
    cap_drop:
      - ALL
    cap_add:
      - NET_BIND_SERVICE
    security_opt:
      - no-new-privileges:true
    networks:
      - frontend
```

```

    - backend
ports:
  - "192.0.2.10:80:80"
  - "192.0.2.10:443:443"
volumes:
  - ./certs:/etc/pki/tls/certs:ro
  - ./private:/etc/pki/tls/private:ro
  - ./ca:/etc/pki/ca-trust/source/anchors:ro
  - ./logs:/var/log/httpd
  - ./modsec_audit:/var/log/httpd/modsec_audit
  - ./banned-hosts:/etc/httpd/conf.d/banned-hosts:ro
healthcheck:
  test: ["CMD", "wget", "-q", "--spider", "--no-check-certificate",
"https://localhost/server/api/core/health"]
  interval: 30s
  timeout: 5s
  retries: 3
  start_period: 10s

networks:
  frontend:
    driver: bridge
  backend:
    driver: bridge
    internal: true

```

Runtime controls that matter:

Control	Why It Matters
user: "48:48"	Enforces non-root at runtime – even if Dockerfile USER is missing
read_only: true	Root FS immutable – container can't modify binaries or configs
tmpfs mounts	Writable dirs on volatile memory – /run/httpd (PID, sockets), /tmp (ModSecurity), /var/cache/httpd (proxy)
cap_drop: ALL + cap_add: NET_BIND_SERVICE	Keeps <i>only</i> port binding. Default Docker gives CHOWN, DAC_OVERRIDE, FOWNER, SETGID, SETUID, KILL...
no-new-privileges:true	Kernel refuses setuid/setgid exec – even if setuid binary exists
backend: internal: true	Backend containers (Tomcat, Angular) cannot reach internet – only httpd and each other
ports: "192.0.2.10:80:80"	Docker only publishes on 192.0.2.10. Traffic to other host IPs never reaches container
volumes: :ro	Certs, keys, CA bundle, banned-hosts read-only. Key rotation = replace on host, SIGHUP Apache

Rewrite Rules: Host Validation, Bot Blocking, fail2ban

```
# Block AI scraper bots BEFORE cookie challenge
SetEnvIfNoCase User-Agent "(ChatGPT-User|GPTBot|ClaudeBot|anthropic-
ai|Amazonbot|ShapBot|Bytespider|CensysInspect|PerplexityBot|Omgilibot|Semrush
Bot|AhrefsBot)" bad_bot_ua
RewriteCond %{ENV:bad_bot_ua} ^1$
RewriteRule .* - [F]

# Host header validation
RewriteCond %{HTTP_HOST}
!^(lib\.example\.org|localhost|127\.0\.0\.1|192\.0\.2\.10|10\.0\.0\.50)$ [NC]
RewriteCond %{HTTP_HOST} !^10\.0\.0\. [NC]
RewriteRule .? - [F]

# Block TRACE/TRACK
RewriteCond %{REQUEST_METHOD} ^(TRACE|TRACK)
RewriteRule .* - [F]

# Require HTTP/1.1 or 2.0
RewriteCond %{THE_REQUEST} !HTTP/(1\.1|2\.0)$
RewriteRule .* - [F]

# Block dot files
RewriteCond %{SCRIPT_FILENAME} -d
RewriteCond %{SCRIPT_FILENAME} -f
RewriteRule "^(|/)\." - [F]

# Banned hosts map (updated by fail2ban action)
RewriteMap hosts-deny txt:/etc/httpd/conf.d/banned-hosts
RewriteCond ${hosts-deny:%{REMOTE_ADDR}|NOT-FOUND} !=NOT-FOUND [OR]
RewriteCond ${hosts-deny:%{REMOTE_HOST}|NOT-FOUND} !=NOT-FOUND
RewriteRule ^ /why-am-i-banned.html [L,PT]
```

Important: The production fail2ban configuration uses `geohostsdeny.conf` action (firewall-cmd/ipset) for banning at the network level, and a separate custom action would be needed to write to the banned-hosts map file for the RewriteMap integration shown above. The example `my-blacklist-filter.conf` filter matches plain IP lines that could feed such a map file.

Block	Attack	How It Works
AI bots	GPTBot, ClaudeBot, etc. scraping content	SetEnvIfNoCase matches UA -> 403 <i>before</i> app sees request
Host header validation	Host injection, cache poisoning	Only known hostnames allowed. Host: <code>evil.com</code> = 403

Block	Attack	How It Works
TRACE/TRACK	Cross-site tracing (XST)	TRACE echoes request including HttpOnly cookies
HTTP/1.1+ only	Protocol downgrade, smuggling	HTTP/1.0/0.9 lack Host header, chunked encoding
Dot files	/.git/config, /.env, /.aws/credentials	Path segment starting with . that resolves to real file = 403
	Banned hosts map	fail2ban integration

The RewriteMap trick (conceptual): A custom fail2ban action would write new.banned-hosts, then mv new.banned-hosts banned-hosts. mv is atomic – Apache never sees a partial file. Production currently bans at network layer via ipset.

10 Verification Commands: Run After Every Deploy

1. Non-root process

```
docker exec httpd-hardened ps aux | head -1
# apache      1  0.0  0.2  12345  6789 ?        Ss   10:00   0:00 httpd -
DFOREGROUND
```

2. Bind address

```
docker exec httpd-hardened netstat -tlnp | grep :80
# tcp        0  0  192.0.2.10:80  0.0.0.0:*    LISTEN  1/httpd
```

3. Host header rejection

```
curl -H "Host: evil.com" -k https://192.0.2.10/
# 403 Forbidden
```

4. TRACE blocked

```
curl -X TRACE -k https://192.0.2.10/
# 403 Forbidden
```

5. Directory listing blocked

```
curl -k https://192.0.2.10/some/path/
# 403 Forbidden (not 200 with file list)
```

6. Sensitive file blocked

```
curl -k https://192.0.2.10/.env
# 403 Forbidden
```

7. TLS version enforcement

```
openssl s_client -connect 192.0.2.10:443 -tls1_1
# handshake failure
openssl s_client -connect 192.0.2.10:443 -tls1_2
# CONNECTED(00000003) -- success
```

```
# 8. ModSecurity blocks SQLi
curl -k "https://192.0.2.10/search?q=1'OR'1'='1"
# 403 Forbidden (rule 942100)

# 9. CSP header present
curl -k -I https://192.0.2.10/ | grep -i content-security-policy
# content-security-policy: default-src 'self' ...

# 10. fail2ban integration
# Trigger 5 failed client cert auths -> check banned-hosts map updated -> 6th
request gets /why-am-i-banned.html
```

Put these in CI/CD. If any returns unexpected, the pipeline fails.

What This Means for Your Strategy

Four posts. Four layers. One strategy:

1. **Container architecture** – Blast radius reduction (non-root, read-only, single IP)
2. **Server config** – Policy enforcement (default deny, explicit allow, shut up)
3. **Transport & payload** – Browser enforcement (headers, TLS, WAF with surgical exclusions)
4. **Runtime & verification** – Kernel enforcement (capabilities, networks, atomic fail2ban, CI/CD tests)

Each layer assumes the one before it *will fail*. That's not pessimism – that's **operational realism**. The container escapes? Non-root shell, read-only FS, dropped caps. The directory rules miss something? WAF catches the payload. The WAF has a false negative? fail2ban blocks the IP. The attacker rotates IPs? Host header validation stops the request at the edge.

You don't build this because you're paranoid. You build it because **incident response at 2 AM is a lot shorter when the blast radius is a single container**.

The Habit That Pays Off

Treat every config file as code. Version-control your `httpd.conf`, `ssl.conf`, `mod_security.conf`, `mod_qos.config`, and Dockerfile together. When the baseline updates or a CVE drops, you change one file, rebuild, redeploy – no manual surgery, no drift, no “it works on my machine.” The container is the documentation.

Series Complete

That's the Hardening Apache HTTPD in Docker series. Four posts, one strategy, zero excuses.

If you implement even half of this, your next incident review starts with “the attack was contained at the web tier” instead of “how did they reach the database?”

Questions? Disagreements? War stories? Drop a comment. I read every one.

Appendix: CompTIA Security+ SY0-701 CEU Mapping

This appendix maps each technique in this post to CompTIA Security+ exam objectives for CEU submission.

Technique Implemented	Security+ Domain	SY0-701 Objective	Rationale
<code>read_only: true</code> root filesystem	Domain 3: Security Architecture	3.2: Implement secure system design	Immutable container runtime prevents tampering
<code>cap_drop: ALL</code> + <code>cap_add: NET_BIND_SERVICE</code>	Domain 3: Security Architecture	3.2: Implement secure system design	Least privilege capabilities, removes dangerous defaults
<code>no-new-privileges:true</code>	Domain 3: Security Architecture	3.2: Implement secure system design	Prevents privilege escalation via setuid binaries
<code>backend: internal: true</code> network	Domain 3: Security Architecture	3.4: Implement secure network architecture	Backend isolation, no internet egress
Bind to specific IP (192.0.2.10)	Domain 3: Security Architecture	3.4: Implement secure network architecture	Network segmentation, reduces attack surface
Read-only volume mounts for certs/keys	Domain 3: Security Architecture	3.5: Implement PKI/certificate management	Key rotation without container rebuild
Host header validation rewrite	Domain 3: Security Architecture	3.4: Implement secure network architecture	Prevents host header injection, cache poisoning
AI bot blocking via User-Agent	Domain 1: Threats/Vulns/Mitigat	1.4: Analyze indicators of malicious activity	Automated threat

Technique Implemented	Security+ Domain	SY0-701 Objective	Rationale
	ions		intelligence at edge
RewriteMap atomic fail2ban integration	Domain 4: Security Operations	4.5: Implement automation/orchestration	Real-time IP blocking without race conditions
HTTP/1.1+ only enforcement	Domain 2: Security Architecture	2.3: Implement application security	Prevents protocol downgrade/smuggling
Dot file blocking (/ .git, / .env)	Domain 2: Security Architecture	2.3: Implement application security	Prevents sensitive file exposure
CI/CD verification checklist	Domain 4: Security Operations	4.5: Implement automation/orchestration	Automated compliance validation on deploy

Primary Domain Coverage: Domain 3 (Architecture & Engineering) – ~50% | Domain 2 (Architecture) – ~25% | Domain 4 (Operations) – ~20% | Domain 1 (Threats) – ~5%

Total Security+ Mapping: >=50% of content maps to SY0-701 objectives